

Monitor Class in C# with Examples

In this article, I will discuss **How to Protect Shared Resources in Multithreading using Monitor Class in C#** with Examples. Please read our previous article discussing [How to Protect shared Resources using Lock in C#](#) from Concurrent Access with Examples. As part of this article, we are going to discuss the following pointers.

1. [Understanding the Monitor Class in C#](#)
2. [Example to understand Monitor Class in C# to Protect Shared Resource from Concurrent Access](#)
3. [How does the Monitor Class work in C#?](#)
4. [Example to Understand Monitor.Enter\(lockObject, ref IslockTaken\) Method in C#](#)
5. [Example to Understand TryEnter\(Object, TimeSpan, Boolean\) Method of Monitor Class in C#](#)
6. [Example to Understand Wait\(\) and Pulse\(\) Methods of Monitor Class in C#](#)
7. [Difference Between Monitor and Lock in C#](#)
8. [Limitations of Locks and Monitors in C#](#)

We have already discussed that both Monitor and Lock are used to provide thread safety to a shared resource in a multithreaded application in C#. In our previous article, we have seen how to use the lock mechanism to achieve thread safety in a multi-thread environment. Now, let us proceed and try to understand the Monitor Class and its methods in detail to understand how to protect the shared resource using the monitor class in C# with Examples.

Understanding the Monitor Class in C#:

According to [Microsoft](#), the Monitor Class in C# Provides a mechanism that synchronizes access to objects. Let us simplify the above definition. In simple words, we can say that, like the lock, we can also use this Monitor Class to protect the shared resources in a multi-threaded environment from concurrent access. This can be done by acquiring an exclusive lock on the object so that only one thread can enter the critical section at any given point in time.

The Monitor class is a static class belonging to the System.Threading namespace. As a static class, it provides a collection of static methods, as shown in the image below. Using these static methods, we can provide synchronized access to the critical section associated with a particular object.

```

namespace System.Threading
{
    ...public static class Monitor
    {
        ...public static void Enter(object obj);
        ...public static void Enter(object obj, ref bool lockTaken);
        ...public static void Exit(object obj);
        ...public static bool IsEntered(object obj);
        ...public static void Pulse(object obj);
        ...public static void PulseAll(object obj);
        ...public static bool TryEnter(object obj, TimeSpan timeout);
        ...public static void TryEnter(object obj, int millisecondsTimeout, ref bool lockTaken);
        ...public static void TryEnter(object obj, ref bool lockTaken);
        ...public static bool TryEnter(object obj);
        ...public static bool TryEnter(object obj, int millisecondsTimeout);
        ...public static void TryEnter(object obj, TimeSpan timeout, ref bool lockTaken);
        ...public static bool Wait(object obj, int millisecondsTimeout, bool exitContext);
        ...public static bool Wait(object obj, TimeSpan timeout, bool exitContext);
        ...public static bool Wait(object obj, int millisecondsTimeout);
        ...public static bool Wait(object obj, TimeSpan timeout);
        ...public static bool Wait(object obj);
    }
}

```

Let's understand the roles and responsibilities of each method in the Monitor class. The following is the list of important methods in the Monitor class.

1. **Enter():** When we invoke the Enter method of the Monitor class, it acquires an exclusive lock on the specified object. This also marks the beginning of a critical section or the beginning of a shared resource.
2. **Exit():** When the Monitor class's Exit method is invoked, it releases the lock on the specified object. This marks the end of a critical section or the end of the shared resource protected by the locked object.
3. **Pulse():** When the Pulse method is invoked of the Monitor class, it sends a signal to a thread in the waiting queue of a change in the locked object's state.
4. **Wait():** When the Monitor class's Wait method is invoked, it releases the lock on an object and blocks the current thread until it reacquires the lock.
5. **PulseAll():** When the Monitor class's PulseAll method is invoked, it sends signals to all waiting threads about a change in the locked object's state.
6. **TryEnter():** When we invoke the Monitor class's TryEnter method, it attempts to acquire an exclusive lock on the specified object.

Note: If this is not clear at the moment, don't worry. We will try to understand all the above methods with examples, and you will also see that many overloaded versions of the above methods are available.

Example to understand Monitor Class in C# to Protect Shared Resource from Concurrent Access:

The following is the syntax for using the Enter and Exit methods of the Monitor class to protect a shared resource in a multithreaded environment from concurrent access in C#. All the methods of the Monitor class are static. So, you can see here that we are accessing the Enter and Exit methods using the class name, i.e., Monitor.

```

object lockObject = new object();

try
{
    //Trying to Enter Into the Critical Section

    Monitor.Enter(lockObject);
    //Access the Shared Resource Here
}
finally
{
    //Release the lock or Exit from the Critical Section
    Monitor.Exit(lockObject);
}

```

Let us see an example to understand how to use the Monitor class Enter and Exit method to protect a shared resource in a multithreaded environment in C# from concurrent access. In the example below, we have one shared resource, and we are accessing that resource concurrently using three different threads. Then, we used the Monitor class Enter and Exit Methods to protect the critical section code. In this case, all three threads will try to acquire an exclusive lock, but at any given point in time, only one thread gets an exclusive lock and will enter into the critical section, and all other threads will wait until the thread releases the lock.

```

using System;
using System.Threading;
namespace MonitorDemo
{
    class Program
    {
        private static readonly object lockPrintNumbers = new object();
        public static void PrintNumbers()
        {
            Console.WriteLine(Thread.CurrentThread.Name + " Trying to enter in
critical section");

            try
            {
                Monitor.Enter(lockPrintNumbers);
                Console.WriteLine(Thread.CurrentThread.Name + " Entered into 1
section");

                for (int i = 0; i < 5; i++)
                {
                    Thread.Sleep(100);
                    Console.Write(i + ",");
                }
                Console.WriteLine();
            }
        }
    }
}

```

```

    }
    finally
    {
        Monitor.Exit(lockPrintNumbers);
        Console.WriteLine(Thread.CurrentThread.Name + " Exit from critical
section");
    }
}
static void Main(string[] args)
{
    Thread[] Threads = new Thread[3];
    for (int i = 0; i < 3; i++)
    {
        Threads[i] = new Thread(PrintNumbers)
        {
            Name = "Child Thread " + i
        };
    }
    foreach (Thread t in Threads)
    {
        t.Start();
    }
    Console.ReadLine();
}
}
}

```

Output:

```

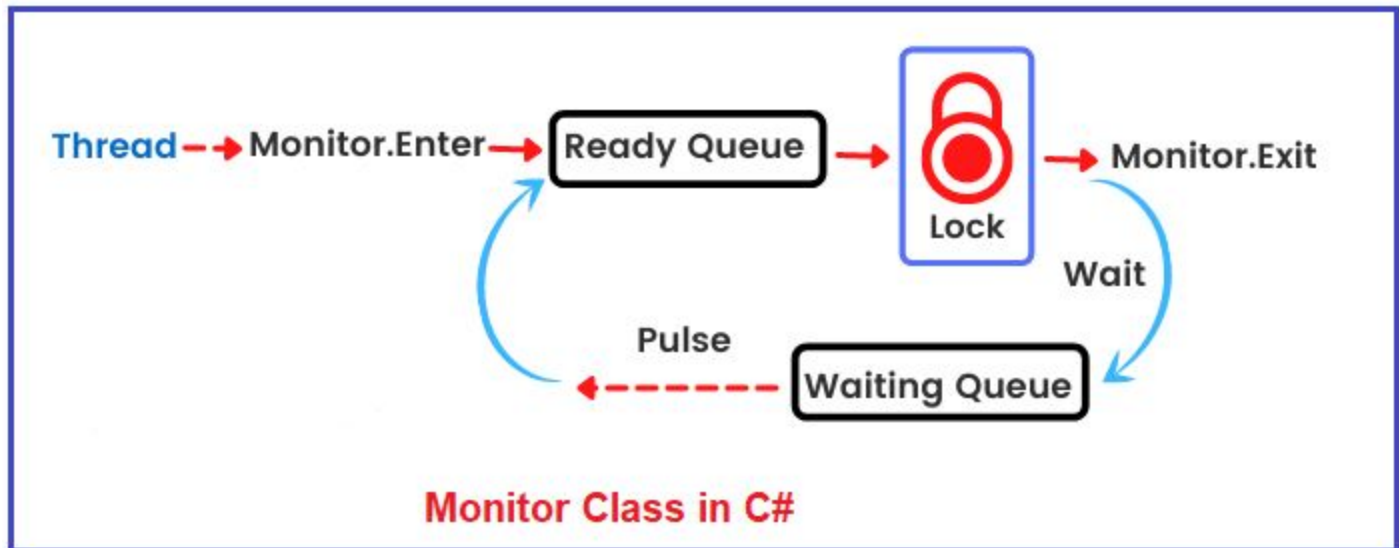
Child Thread 0 Trying to enter into the critical section
Child Thread 0 Entered into the critical section
Child Thread 2 Trying to enter into the critical section
Child Thread 1 Trying to enter into the critical section
0,1,2,3,4,
Child Thread 0 Exit from critical section
Child Thread 2 Entered into the critical section
0,1,2,3,4,
Child Thread 1 Entered into the critical section
Child Thread 2 Exit from critical section
0,1,2,3,4,
Child Thread 1 Exit from critical section

```

How does the Monitor Class work in C#?

The Monitor class in C# provides a wait-based synchronization mechanism that allows only one thread to access the critical section code at a time to avoid the race condition. All the other threads must wait and halt the execution until the locked object is released.

To understand how the Monitor class works in C#, please have a look at the following diagram. As shown in the image below, as soon as a thread executes the Enter method of the Thread class, it will be in the Ready Queue, and in the same way, many threads can be in the Ready Queue. Then, one of the threads from the Ready Queue will acquire an Exclusive Lock on the Object and will enter inside the Critical Section and execute the code, and at that time, no other threads can get a chance to enter into the Critical Section. When we execute the Exit method of the thread class, the currently executing thread will move into the waiting queue and send one signal to the threads in the ready queue. One of the Threads from the Ready queue will acquire the lock, and enter the Critical section, and start executing the code of the Critical Section. This is how the Monitor class works in C#.



Monitor.Enter(lockObject, ref IslockTaken) Method in C#:

Let us understand the other overloaded version of the Enter method. The **Monitor.Enter(lockObject, ref IslockTaken)** acquires an exclusive lock on the specified object. It then automatically sets a value that indicates whether the lock was taken or not. The second parameter, which is a Boolean parameter, returns true if the lock is acquired; otherwise, it returns false. The syntax for using this overloaded version is given below.

```

private static readonly object lockObject = new object();
bool IsLockTaken = false;

try
{
    //Trying to Enter into the Critical Section
    Monitor.Enter(lockObject, ref IsLockTaken);
    if(IsLockTaken)
    {
        //Entered Into the Critical Section
        //Access the Shared Resource
    }
}
finally
{
    //Exit From Critical Section
    if (IsLockTaken)
    {
        Monitor.Exit(lockObject);
    }
}

```

The following example shows how to use **Enter(lockObject, ref IslockTaken)** method of the Monitor class in C#. The following example is the same as the previous example, except here, we are using the overloaded version of the Enter method, which takes two parameters. The second boolean parameter specifies whether the thread acquires a lock or not, true indicates that it acquires a lock on the object and false indicates that it does not acquire a lock on the object and again in the finally block we are checking the boolean value and accordingly we are releasing the lock.

```

using System;
using System.Threading;
namespace MonitorDemo
{
    class Program
    {
        private static readonly object lockPrintNumberst = new object();
        public static void PrintNumbers()
        {
            Console.WriteLine(Thread.CurrentThread.Name + " Trying to enter in
critical section");
            bool IsLockTaken = false;

            try
            {
                Monitor.Enter(lockPrintNumberst, ref IsLockTaken);

```

```

        if(IsLockTaken)
        {
            Console.WriteLine(Thread.CurrentThread.Name + " Entered ir
critical section");
            for (int i = 0; i < 5; i++)
            {
                Thread.Sleep(100);
                Console.Write(i + ",");
            }
            Console.WriteLine();
        }
    }
    finally
    {
        if (IsLockTaken)
        {
            Monitor.Exit(lockPrintNumberst);
            Console.WriteLine(Thread.CurrentThread.Name + " Exit from
section");
        }
    }
}
static void Main(string[] args)
{
    Thread[] Threads = new Thread[3];
    for (int i = 0; i < 3; i++)
    {
        Threads[i] = new Thread(PrintNumbers)
        {
            Name = "Child Thread " + i
        };
    }
    foreach (Thread t in Threads)
    {
        t.Start();
    }
    Console.ReadLine();
}
}
}

```

Output:

```
Child Thread 0 Trying to enter into the critical section
Child Thread 0 Entered into the critical section
Child Thread 1 Trying to enter into the critical section
Child Thread 2 Trying to enter into the critical section
0,1,2,3,4,
Child Thread 0 Exit from critical section
Child Thread 1 Entered into the critical section
0,1,2,3,4,
Child Thread 1 Exit from critical section
Child Thread 2 Entered into the critical section
0,1,2,3,4,
Child Thread 2 Exit from critical section
```

Example to Understand TryEnter(Object, TimeSpan, Boolean) Method of Monitor Class in C#:

This method attempts to acquire an exclusive lock on the specified object for a specified amount of time. It automatically sets a value that indicates whether the lock was taken or not. The syntax for using the **TryEnter(Object, TimeSpan, Boolean)** Method of Monitor Class in C# is given below.

```
var lockObj = new object();
var timeout = TimeSpan.FromMilliseconds(500);
bool lockTaken = false;

try {
    Monitor.TryEnter(lockObj, timeout, ref lockTaken);
    if (lockTaken) {
        // The critical section.
    }
    else {
        // The lock was not acquired.
    }
}
finally {
    // Ensure that the lock is released.
    if (lockTaken) {
        Monitor.Exit(lockObj);
    }
}
```

For a better understanding, please have a look at the example below, which shows how to use the TryEnter(Object, TimeSpan, Boolean) Method of the Monitor Class in C#. In the example below, we specified the timeout as 1000 milliseconds, or you can say 1 second. If the thread does not acquire the lock within 1 second, it will not enter the critical section.

```

using System;
using System.Threading;
namespace MonitorDemo
{
    class Program
    {
        private static readonly object lockPrintNumbers = new object();
        public static void PrintNumbers()
        {
            TimeSpan timeout = TimeSpan.FromMilliseconds(1000);
            bool lockTaken = false;
            try
            {
                Console.WriteLine(Thread.CurrentThread.Name + " Trying to enter
critical section");
                Monitor.TryEnter(lockPrintNumbers, timeout, ref lockTaken);
                if (lockTaken)
                {
                    Console.WriteLine(Thread.CurrentThread.Name + " Entered in
critical section");
                    for (int i = 0; i < 5; i++)
                    {
                        Thread.Sleep(100);
                        Console.Write(i + ",");
                    }
                    Console.WriteLine();
                }
                else
                {
                    // The lock was not acquired.
                    Console.WriteLine(Thread.CurrentThread.Name + " Lock was not
acquired");
                }
            }
            finally
            {
                // To Ensure that the lock is released.
                if (lockTaken)
                {
                    Monitor.Exit(lockPrintNumbers);
                    Console.WriteLine(Thread.CurrentThread.Name + " Exit from
section");
                }
            }
        }
        static void Main(string[] args)
        {
            Thread[] Threads = new Thread[3];
            for (int i = 0; i < 3; i++)
            {
                Threads[i] = new Thread(PrintNumbers)

```

```

        {
            Name = "Child Thread " + i
        };
    }
    foreach (Thread t in Threads)
    {
        t.Start();
    }
    Console.ReadLine();
}
}
}

```

When you run the above code, you will get the following output. The output may vary on your machine. As you can see, all three threads try to acquire a lock on the object within 1 second. Two of the three threads acquire an exclusive lock on the object, while one is unable to acquire an exclusive lock, and hence, that thread will not enter the critical section.

```

Child Thread 1 Trying to enter into the critical section
Child Thread 1 Entered into the critical section
Child Thread 0 Trying to enter into the critical section
Child Thread 2 Trying to enter into the critical section
0,1,2,3,4,
Child Thread 1 Exit from critical section
Child Thread 2 Entered into the critical section
0,1,2,3,Child Thread 0 Lock was not acquired
4,
Child Thread 2 Exit from critical section

```

Example to Understand Wait() and Pulse() Methods of Monitor Class in C#:

The Wait() Method of Monitor Class is used to Release the lock on an object in order to permit other threads to lock and access the object. The calling thread waits while another thread accesses the object. The Pulse signals are used to notify waiting threads about changes to a locked object's state.

Let us understand this with one real-time example. Our business requirement is to print the even and Odd number sequence using two different threads. One thread will print the even numbers, and another thread will print the odd numbers.

Thread T1: 0,2,4,6,8...

Thread T2: 1,3,5,7,9...

Output: 0,1,2,3,4,5,6,7,8,9...

To solve the above problem, let us use the signaling mechanism using the Monitor Class Wait() and Pulse() Methods in C#. In the following example, we use the Monitor.Wait() method to make the thread waiting and Monitor.Pulse() method is used to signal other threads. The process is as follows:

1. First, the Even thread will start to print the number on the console.
2. Then, the Even thread will signal the Odd thread to be ready to print the number using the Monitor.Pulse() method.

3. Then, the Event thread will call the Monitor.Wait() method, which will allow the current thread to block and the Odd thread to start execution.
4. The Odd Thread will also do the same thing.
5. The Odd thread will start to print the number on the console.
6. Then, the Odd thread will signal the Even thread to be ready to print the number using the Monitor.Pulse() method.
7. Then the Odd thread will call the Monitor.Wait() method, which will allow the current thread to block and allow the Even thread to start execution.
8. The same process is going on.

Since both the Odd and Even threads share the same console window to print the number, we need to put a lock on the console IO. We want the sequence to start with the even number, so the Even thread must run first. Once we start the Even thread, we need to pause for a moment before starting the Odd thread using the Sleep() method of the Thread class in C# to avoid any chance of starting the Odd thread first.

```
using System;
using System.Threading;
namespace odd_even_sequence
{
    class Program
    {
        //Upto the limit numbers will be printed on the Console
        const int numberLimit = 20;
        static readonly object _lockMonitor = new object();
        static void Main(string[] args)
        {
            Thread EvenThread = new Thread(PrintEvenNumbers);
            Thread OddThread = new Thread(PrintOddNumbers);
            //First Start the Even thread.
            EvenThread.Start();
            //Puase for 10 ms, to make sure Even thread has started
            //or else Odd thread may start first resulting different sequence.
            Thread.Sleep(100);
            //Next, Start the Odd thread.
            OddThread.Start();
            //Wait for all the childs threads to complete
            OddThread.Join();
            EvenThread.Join();
            Console.WriteLine("\nMain method completed");
            Console.ReadKey();
        }
        //Printing of Even Numbers Function
        static void PrintEvenNumbers()
        {
            try
            {
                //Implement lock as the Console is shared between two threads
                Monitor.Enter(_lockMonitor);
                for (int i = 0; i <= numberLimit; i = i + 2)
                {
                    //Printing Even Number on Console)
                }
            }
        }
    }
}
```

```

        Console.Write($"{i} ");
        //Notify Odd thread that I'm done, you do your job
        //It notifies a thread in the waiting queue of a change in
        //locked object's state.
        Monitor.Pulse(_lockMonitor);
        //I will wait here till Odd thread notify me
        //Monitor.Wait(monitor);
        //Without this logic application will wait forever
        bool isLast = false;
        if (i == numberLimit)
        {
            isLast = true;
        }
        if (!isLast)
        {
            //I will wait here till Odd thread notify me
            //Releases the lock on an object and blocks the current
            //until it reacquires the lock.
            Monitor.Wait(_lockMonitor);
        }
    }
}
finally
{
    //Release the lock
    Monitor.Exit(_lockMonitor);
}
}
//Printing of Odd Numbers Function
static void PrintOddNumbers()
{
    try
    {
        //Hold lock as the Console is shared between two threads
        Monitor.Enter(_lockMonitor);
        for (int i = 1; i <= numberLimit; i = i + 2)
        {
            //Printing the odd numbers on the console
            Console.Write($"{i} ");
            //Notify Even thread that I'm done, you do your job
            Monitor.Pulse(_lockMonitor);
            // I will wait here till even thread notify me
            // Monitor.Wait(monitor);
            // without this logic application will wait forever
            bool isLast = false;
            if (i == numberLimit - 1)
            {
                isLast = true;
            }
            if (!isLast)
            {

```

```

        //I will wait here till Even thread notify me
        Monitor.Wait(_lockMonitor);
    }
}
finally
{
    //Release lock
    Monitor.Exit(_lockMonitor);
}
}
}
}

```

Output:

```

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
Main method completed

```

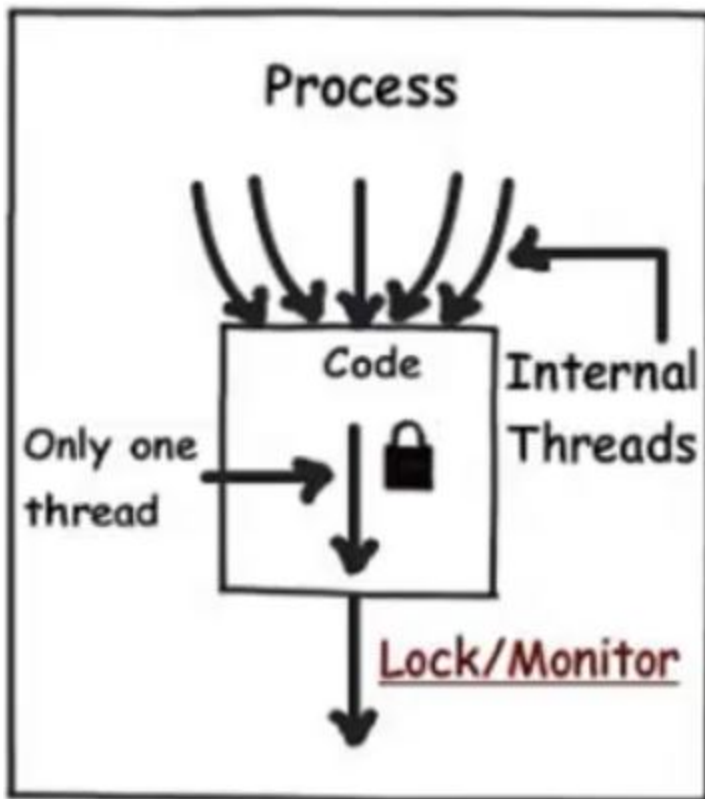
Difference Between Monitor and Lock in C#

The Difference between monitor and lock in C# is that lock internally wraps the Enter and Exit methods in a try...finally block with exception handling. For the Monitor class in C#, we need to use the try and finally block explicitly to release the lock properly. So, Lock = Monitor + try-finally.

The lock provides the basic functionality to acquire an exclusive lock on a synchronized object. But, If you want more control to implement advanced multithreading solutions using TryEnter(), Wait(), Pulse(), and PulseAll() methods, then the Monitor class is your option.

Limitations of Locks and Monitors in C#:

Locks and Monitors help us ensure that our code is thread-safe. That means when we run our code in a multi-threaded environment, we don't end up with inconsistent results. For a better understanding, please have a look at the image below.



However, there are some limitations to locks and monitors. The locks and monitors ensure thread safety for threads that are in process, i.e., the threads that are generated by the application itself, i.e., Internal Threads. But if the threads come from external applications (Out-Process) or External Threads, then Locks and Monitors have no control over them. So, in a situation like this, we need to use Mutex. In our next article, we will discuss Mutex.

Monitor class Methods in Detail:

Let us understand the Roles and Responsibilities of each method of the Monitor class according to Microsoft.

1. **Enter(object obj):** This method acquires an exclusive lock on the specified object. It takes one object parameter to acquire the monitor lock. If the parameter obj is null, it will throw an ArgumentException.
2. **Enter(object obj, ref bool lockTaken):** This method also acquires an exclusive lock on the specified object and atomically sets a value that indicates whether the lock was taken. Here, the parameter obj specifies the object on which to wait. The parameter lockTaken specifies the result of the attempt to acquire the lock passed by reference. The input must be false. The output is true if the lock is acquired; otherwise, the output is false. The output is set even if an exception occurs during the attempt to acquire the lock. If no exception occurs, the output of this method will always be true. It will throw ArgumentException if the input to lockTaken is true. It will throw ArgumentException if the obj parameter is null.

TryEnter Methods:

There are six overloaded versions of the TryEnter method available in the Monitor class. They are as follows:

1. **public static bool TryEnter(object obj, TimeSpan timeout):** Attempts, for the specified amount of time, to acquire an exclusive lock on the specified object.

2. **public static void TryEnter(object obj, int millisecondsTimeout, ref bool lockTaken):** Attempts, for the specified number of milliseconds, to acquire an exclusive lock on the specified object and atomically sets a value that indicates whether the lock was taken.
3. **public static void TryEnter(object obj, ref bool lockTaken):** Attempts to acquire an exclusive lock on the specified object and atomically sets a value that indicates whether the lock was taken.
4. **public static bool TryEnter(object obj):** Attempts to acquire an exclusive lock on the specified object.
5. **public static bool TryEnter(object obj, int millisecondsTimeout):** Attempts, for the specified number of milliseconds, to acquire an exclusive lock on the specified object.
6. **public static void TryEnter(object obj, TimeSpan timeout, ref bool lockTaken):** Attempts, for the specified amount of time, to acquire an exclusive lock on the specified object and atomically sets a value that indicates whether the lock was taken.

All these methods are also used to acquire an exclusive lock on the specified object. Further, if you notice, all these methods return a type of bool. So, the TryEnter() method returns true if the current thread acquires the lock; otherwise, false. The following are the parameters used in the TryEnter method.

1. **object obj:** All the six overloaded versions take one object type parameter which specifies the object on which to acquire the lock. If the object parameter this method takes is null, then it will throw ArgumentNullException.
2. **TimeSpan timeout:** Some TryEnter() methods take TimeSpan timeout as a parameter, and this parameter specifies a System.TimeSpan represents the amount of time to wait for the lock. A value of -1 millisecond specifies an infinite wait. It will throw ArgumentOutOfRangeException if the value of timeout in milliseconds is negative and is not equal to System.Threading.Timeout.Infinite (-1 millisecond), or is greater than System.Int32.MaxValue.
3. **int millisecondsTimeout:** Again, two overloaded versions take int millisecondsTimeout as a parameter and this parameter specifies the number of milliseconds to wait for the lock. It will throw ArgumentOutOfRangeException if millisecondsTimeout is negative, and not equal to System.Threading.Timeout.Infinite.
4. **ref bool lockTaken:** Also three overloaded versions take ref bool lockTaken as a parameter and this parameter specifies the result of the attempt to acquire the lock, passed by reference. The input must be false. The output is true if the lock is acquired; otherwise, the output is false. The output is set even if an exception occurs during the attempt to acquire the lock. It will ArgumentException if the input to lockTaken is true.

Note: Both Enter and TryEnter methods are used to acquire an exclusive lock for an object. This action marks the beginning of a critical section. No other thread can enter the critical section unless it executes the instructions in the critical section using a different locked object.

Wait Methods of Monitor Class in C#:

There are five overloaded versions of the Wait method available in the Monitor class. They are as follows:

1. **public static bool Wait(object obj):** It Releases the lock on an object and blocks the current thread until it reacquires the lock.
2. **public static bool Wait(object obj, TimeSpan timeout):** Releases the lock on an object and blocks the current thread until it reacquires the lock. If the specified time-out interval elapses, the thread enters the ready queue.
3. **public static bool Wait(object obj, int millisecondsTimeout):** It releases the lock on an object and blocks the current thread until it reacquires the lock. If the specified time-out interval elapses, the thread enters the ready queue.

4. **public static bool Wait(object obj, TimeSpan timeout, bool exitContext):** It releases the lock on an object and blocks the current thread until it reacquires the lock. If the specified time-out interval elapses, the thread enters the ready queue. Optionally exits the synchronization domain for the synchronized context before the wait and reacquires the domain afterward.
5. **public static bool Wait(object obj, int millisecondsTimeout, bool exitContext):** It releases the lock on an object and blocks the current thread until it reacquires the lock. If the specified time-out interval elapses, the thread enters the ready queue. This method also specifies whether the synchronization domain for the context (if in a synchronized context) is exited before the wait and reacquired afterward.

All these Wait Methods are used to release the lock on an object and block the current thread until it reacquires the lock. All these methods return type is boolean. So, these methods return true if the call is returned because the caller reacquired the lock for the specified object. This method does not return if the lock is not reacquired. Following are the parameters used in the Wait method.

1. **object obj:** The object on which to wait. If the obj parameter is null, it will throw an ArgumentNullException.
2. **TimeSpan timeout:** A System.TimeSpan represents the amount of time to wait before the thread enters the ready queue. It will throw ArgumentOutOfRangeException if the value of the timeout parameter in milliseconds is negative and does not represent System.Threading.Timeout.Infinite (-1 millisecond), or is greater than System.Int32.MaxValue.
3. **int millisecondsTimeout:** The number of milliseconds to wait before the thread enters the ready queue. If the value of the millisecondsTimeout parameter is negative and not equal to System.Threading.Timeout.Infinite, it will throw ArgumentOutOfRangeException.
4. **bool exitContext:** true to exit and reacquire the synchronization domain for the context (if in a synchronized context) before the wait; otherwise, false.
5. **ref bool lockTaken:** The result of the attempt to acquire the lock, passed by reference. The input must be false. The output is true if the lock is acquired; otherwise, the output is false. The output is set even if an exception occurs during the attempt to acquire the lock.

Note: The Wait methods are used to release the lock on an object and permit other threads to lock and access the object by blocking the current thread until it reacquires the lock. The calling thread waits while another thread accesses the object. Pulse signals are used to notify waiting threads about changes to an object's state.

Pulse and PulseAll Method of Monitor Class in C#:

The following two methods are used to send a signal to one or more waiting threads. The signal notifies a waiting thread that the state of the locked object has changed, and the owner of the lock is ready to release the lock.

1. **Pulse(object obj):** This method notifies a thread in the waiting queue of a change in the locked object's state. The obj parameter specifies the object a thread is waiting for. If the obj parameter is null, then it will throw ArgumentNullException.
2. **PulseAll(object obj):** This method notifies all waiting threads of a change in the object's state. The obj parameter specifies the object that sends the pulse. If the obj parameter is null, then it will throw ArgumentNullException.

Exit():

The Exit method is used to release the exclusive lock from the specified object. This action marks the end of a critical section protected by the locked object.

1. **Exit(object obj):** This method releases an exclusive lock on the specified object. The parameter obj specifies the object on which to release the lock. If the obj parameter is null, it will throw an ArgumentNullException.

IsEntered() Method:

1. **IsEntered(object obj):** This function determines whether the current thread holds the lock on the specified object. The parameter obj specifies the object to test. It returns true if the current thread holds the lock on obj; otherwise, it returns false. If the obj is null, it will throw an ArgumentNullException.

In the next article, I will discuss the [Mutex in C#](#) with Examples. In this article, I try to explain how to protect shared resources in multithreading using the Monitor class from Concurrent Access in C# with examples. I hope you enjoy this article.